PROJECT PLANNING REPORT

TEAM 3: RNAverse

IS Director: *Professor James Deng*

Team Members:

Priyanshu Mittal
Samarjeet Singh
Saif Alam
Aniket Thakur

THE UNIVERSITY OF WINNIPEG

Table of Contents

1. Introduction
   1.1 Project Initiation
   1.2 User Environment
   1.3 Description of the Current System
   1.4 Shortcomings of the Current System
2. Proposed System
   2.1 Objectives and Benefits of the Proposed System
   2.2 Scope of the Proposed System
   2.3 Statement of User Requirements
3. System Architecture
   3.1 High Level Design
      3.1.1 System Overview
      3.1.2 System Architecture
   3.2 Software Components
      3.2.1 Front- End Architecture
      3.2.2 Back- End Architecture
   3.3 Hardware and Hosting Plans
      3.3.1 Deployment Environment (ACS Server)
      3.3.2 Resource Requirements
   3.4 Data Flow and Model Integration
      3.4.1 Input Handling and Preprocessing
      3.4.2 Model Selection
      3.4.3 Output Generation and Visualization
4. Project Teams & Roles
5. Advisability Study
   5.1 Evaluation Criteria
   5.2 Software Options and Comparative Analysis
      5.2.1 Python + Django
      5.2.2 Node.js + Express
      5.2.3 JavaScript (React/Angular)
      5.2.4 Database Options
   5.3 Tool Comparative Analysis
      5.3.1 GitHub
      5.3.2 Jira
      5.3.3 Docker
   5.4 Hardware and Hosting Considerations
   5.5 Recommendations
6. General Design
   6.1 Description
   6.2 Context-Level Process Model
   6.3 Preliminary Data Model
   6.4 Description of Screens/Reports

6.5 Testing Plans
6.6 Conversion Plans
6.7 Training Plans
7. Agile Development Stage
7.1 Timeline and Sprints
7.2 Agile (Scrum) Implementation
7.3 Agile (Scrum) Artifacts
7.4 Supporting Practices
7.5 Continuous Integration and Testing
8. Project Management
8.1 Assumptions
8.2 Change Control Management
8.3 Maintenance Issues
8.4 Project Plan
9. Budget and Resources
10. Appendices
A. DFD for External System
B. DFDs for Proposed System
C. Report Designs
D. Screen Design Wireframes
E. Work Request Form
F. Jira Snapshot
G. ERD
H. Architecture Design Blueprint

1.0 Introduction

Methylation of RNA is an important post-transcriptional modification that significantly affects gene regulation, cellular development and immune responses. Of the various types of RNA methylations, m6A, m7G, and m5C are best researched for their involvement in biological processes and diseases. With the advent of ML and DL in bioinformatics, the prediction of the status of RNA methylation from sequence data is now possible. Nonetheless, non-coding researchers usually face limitations when utilizing ML/DL platforms productively. For the above limitation to be overcome, this project seeks to implement a fully web-based platform where users can make RNA methylation status predictions from RNA sequence inputs directly through pre-trained ML/DL models. This project increases the usability for non-coding biological researchers and facilitates further scientific investigation in RNA biology.

1.1 Project Initiation

The initiative was undertaken by Dr. Sheela Ramanna and Dr. Qian Liu, faculty members from the Department of Applied Computer Science at the University of Winnipeg. It is assisted by Dr. Muhammad Tahir, Postdoctoral Fellow, whose currently ongoing research on hybrid learning models for m6A modifications provided the groundwork for the project.

The team's overall purpose is to build simple, modular, and scalable web-based platform that provides various functions such as:

1) Predict several types of RNA methylation (m6A, m7G, m5C, etc.)
2) Combine pre-trained models like SVM, Random Forest, XGBoost, CNN and MLP.
3) Provide data visualization and downloadable output for researchers.

1.2 User Environment

Web-based access via desktop or laptop

- Compatible with major browsers (Chrome, Firefox, Edge.)
- Optional lightweight cloud access for compute-intensive ML/DL predictions.
- Designed for intuitive navigation, minimal technical setup, and rapid result generation.

Users: Researchers studying RNA methylation who have strong biological knowledge but limited coding or machine-learning skills. These users need an easy way to run predictions without writing code.

Environment: Our web based platform provides one simple, browser-assessible environment, running preliminary on the university's ACS server.

Limits / Restrictions: Users may have limited compute resources and limited time, so the system should be lightweight, responsive, and straightforward to use. Results should be downloadable so users can save and manage data on their own systems.

1.3 Description of the Current System

Currently, there is no system present within the University of Winnipeg to predict RNA methylation through a web platform. While there are external tools like m5CStack or other RNA methylation predictors, they are limited to single input of RNA sequence. Other tools which require programming knowledge exist but are not suitable for users without a programming background.

Function of External Tool Current system:

The system provides various features:

- Uses predefined models trained on specific datasets.
- Generates prediction outputs with probabilities for each sequence.
- Allows users to download prediction results as a file.
- Allows users to specify a Project ID to track their submission.

1.4 Shortcomings of the Current System

The system also comes with drawbacks:

- Existing tools like m5CStack(external application) focus only on one methylation type.
- No support for comparing multiple ML/DL models (SVM, CNN, Random Forest, etc.).
- Missing visualization and batch input support.
- Do not provide flexibility to expand with new methylation types or models.
- Usability support is weak, with few tips, examples, or guidance for non-technical users.
- Many tools accept invalid sequences without feedback, which may cause incorrect predictions or errors.

2.0 Proposed System

The proposed system is a web-based platform that enables users to predict RNA methylation status using pre-trained machine learning and deep learning models. It allows users to input single or multiple RNA sequences, obtain prediction results, and view visual representations to help interpret outcomes. The platform will be developed using Django for the backend (to manage data flow and integrate models) and React for the frontend (to provide an interactive and responsive interface). The system will be hosted on the University of Winnipeg's Applied Computer Science (ACS) servers for accessibility and reliability.

2.1 Objectives and Benefits of the Proposed System

The proposed system aims to provide an accessible, efficient, and modular web-based platform for predicting RNA methylation using pre-trained machine learning and deep learning models.

- **User-Friendly Interface:** Enable researchers to input single or multiple RNA sequences for methylation prediction without requiring programming knowledge.
- **Integrated ML/DL Models:** Incorporate multiple pre-trained models such as SVM, Random Forest, Logistic Regression, MLP, XGBoost, and CNN to generate accurate predictions.
- **Result Visualization:** Display prediction outcomes through intuitive graphical charts and downloadable CSV files for easier interpretation.
- **Multi-Model Comparison:** Allow comparison among different models to identify the one providing the highest prediction probability.
- **Batch Processing:** Support submission of multiple RNA sequences simultaneously to save time and effort.
- **Reliable Hosting:** Deploy the system on the University of Winnipeg's ACS servers to ensure secure, stable, and consistent performance.
- **Maintainability and Scalability:** Use open-source technologies and a modular architecture that supports easy updates, integration of new methylation types, or model enhancements in the future.
- **Dataset Availability:** The system will include the capability to host curated RNA methylation datasets on the website, pending confirmation of dataset availability from the project sponsors and external sources.

2.2 Scope of the Proposed System

In Scope:

- Integration of pre-trained ML and DL models for methylation prediction.
- Input support for single and multiple RNA sequences.
- Results presented in both downloadable CSV format and graphical visualizations.
- Front-end interface using React and backend developed with Django.
- Basic help and documentation features for users.
- Hosting the platform on the University of Winnipeg's ACS servers.

Out of Scope:

- Development of new ML/DL models.
- Prediction of RNA methylation beyond the currently available models.
- User registration or personalized accounts.
- Advanced analytics beyond CSV and visual charts.
- Mobile application development or mobile-friendly optimization.

The scope ensures that the system remains focused on delivering an accessible, model-based RNA methylation prediction tool.

2.3 Statement of User Requirements

Functional Requirements:

- The system must allow users to input one or multiple RNA sequences.
- The system must process inputs and return prediction results using the integrated ML/DL models.
- The system must generate visual charts to represent prediction outcomes.
- The system must allow users to download results in CSV format.
- The system must display appropriate error messages for invalid inputs or processing issues.

Non-Functional Requirements:

- System Performance & Reliability:
  - Response Time: The interface should load within 3 seconds and provide clear progress indicators during prediction processing.
  - Error Handling: The system should gracefully handle invalid inputs and processing errors with descriptive messages that guide users toward resolution.
- User Experience Quality:
  - Learnability: New users should be able to submit their first prediction request within 5 minutes of accessing the platform without external guidance.
  - Efficiency: Researchers should be able to process batches of up to 50 RNA sequences in a single submission with proportional processing time.
  - Help Accessibility: Documentation and examples should be accessible within two clicks from any screen in the application.
- Technical Quality Attributes:
  - Modularity: The codebase should be organized into discrete components (frontend, backend, models) that can be developed and tested independently.
  - Extensibility: The system architecture must accommodate integration of at least three additional methylation types without requiring structural changes.
  - Portability: The application should deploy consistently across development, testing, and production environments using containerization.
- Data Management & Security:
  - Data Integrity: All user submissions and prediction results must maintain accuracy through the complete processing pipeline.
  - Privacy: While no personal user data is collected, sequence data and results should be handled securely and not exposed to unauthorized access.
  - Recoverability: System should maintain functionality even if individual components (like specific ML models) encounter temporary issues.

3.0 System Architecture

The system architecture defines the structural, functional, and deployment design required to support RNA methylation prediction using multiple trained machine learning and deep learning models. The architecture supports modularity, scalability, and the integration of several methylation types.

3.1 High Level Design

3.1.1 System Overview

The platform is designed as a modular web-based system that enables researchers to submit RNA sequences and obtain predicted methylation status using pre-trained models. The platform consists of a web interface, an application layer for processing requests, a model serving layer for running predictions and a back end data layer for temporary storage and output management. The system operates as a layer architecture in which a component performs a distinct function.

- The presentation layer manages sequence submission and displays predicted results.
- The application layer coordinates regulation, preprocessing, methylation type selection, model selection, and inference requests.
- Model layer loads as SVM, RF, LR, MLP, XG Boost and CNN models trained by sponsor team.
- The data layer maintains logs, cached outputs, and downloadable results.

This structure sports additional methylation types and models without requiring changes to the core application flow.

3.1.2 System Architecture

A diagram would depict:

- User interface communicated with the Application Server.
- Application server invoking the Model Serving Engine.
- Feature encoder and preprocessing pipeline.
- Trained models stored in a model repository.
- Data Store for logs and output files.
- Final results returned to the user interface with visualisations and CSV export.

3.2 Software Components

3.2.1 Front End Architecture

Technologies Used:

- HTML, CSS and JavaScript for client side interface.
- React based components for dynamic rendering.
- Charting libraries for visual output (line charts, probability bars).

User Interface Design:

The interface provides fields for sequence entry, methylation type selection, batch submission and results viewing. The design prioritises clarity and reliability to accommodate non technical users.

Input Validation and Error Handling:

Validation checks to ensure that sequences contain only valid RNA characters and also checks to verify length limits, formatting and batch entry correctness. Invalid sequences trigger descriptive error messages.

Visualization and Output Display:

Results are presented through tables, probability indicators, and graphic visuals. Prediction outputs are summarized per sequence and per model.

3.2.2 Back End Architecture

Framework and APIs:

A Python based web framework such as Flask or Django manages routing, request handling and response generation. REST style APIs enable structured communication between the front end and backend services.

Model Integration:

Modules are provided for loading and executing SVM, RF, LR, MLP, XGBoost, and CNN models. Each model is stored with its preprocessing configuration involved through a standardized model interface using FAST APIs.

Data Processing and Prediction Workflow:

The workflow consists of

- Sequence validation.
- Feature extraction and encoding.
- Model invocation based on selected methylation type.
- Aggregation of outputs for visualization.
- Formatting of results for CSV export.

File Handling and CSV Export:

Prediction outputs are generated as downloadable CSV files. Temporary storage maintains results for short retention to support user download.

3.3 Hardware and Hosting Plans

3.3.1 Deployment Environment (ACS server)

The system is intended for deployment on a University of Winnipeg Applied Computer Science (ACS) server for its initial release. The environment supports web hosting, backend execution, and model inference within a controlled academic infrastructure.

To ensure smooth deployment and consistent performance across different environments, the system uses Docker for containerization. Docker packages the complete application, including the web framework, dependencies, and pre-trained models, into isolated containers. This approach eliminates compatibility issues between local and server environments, simplifies updates, and ensures reproducibility of results.

Docker ensures that the entire application, including its dependencies, models, and configurations, runs identically across development and production environments. This reduces deployment errors, simplifies collaboration, and enables quick transfer between local machines and the ACS hosting server.

3.3.2 Resource Requirements

- Multi-core CPU to handle inference operations and concurrent user requests.
- 16–32 GB RAM for loading ML/DL models and batch RNA sequence processing.
- Sufficient storage for model artifacts, log files, and temporary outputs.
- HTTPS-enabled secure network access for data transfer.

- Reverse proxy setup for load distribution and request forwarding.
- Docker-based deployment environment for consistency across systems.

3.4 Data Flow and Model Integration

3.4.1 Input Handling and Preprocessing

When a researcher submits RNA sequences through the web interface, the system first validates the input to ensure data quality. The validation process checks that sequences contain only valid RNA nucleotides A (adenine), U (uracil), C (cytosine), and G (guanine) and meet formatting requirements. Invalid sequences trigger clear error messages to guide users in correcting their input. Once validated, the sequences enter the preprocessing pipeline where they undergo feature extraction.

3.4.2 Model Selection

The system intelligently routes each prediction request to the appropriate pre-trained models based on the user's selected methylation type (such as m6A, m5C, or m7G). When a user chooses a specific methylation type from the dropdown menu, the backend automatically identifies and loads the corresponding machine learning models from the model repository. Each model is stored with its specific configuration and preprocessing requirements. The system maintains a standardized interface that ensures consistent loading and execution across all model types whether its Support Vector Machines (SVM), Random Forests, Convolutional Neural Networks (CNN), or other algorithms. This modular approach allows the platform to seamlessly handle multiple methylation types and makes it easy to add new models in the future without disrupting the core system.

3.4.3 Output Generation and Visualization

After model execution, the system generates comprehensive prediction results including probability scores and classification labels for each RNA sequence. These raw predictions are then formatted into user-friendly outputs that combine both visual and tabular representations. The platform presents results through interactive charts and graphs that help researchers quickly identify methylation patterns and confidence levels. Simultaneously, detailed tabular data shows precise numerical values for each prediction. Users can download complete result sets as CSV files for further analysis in external tools or for inclusion in research publications. The entire data flow concludes with a unified response that delivers both the visualizations and downloadable content back to the user's browser, creating a seamless end-to-end prediction experience.

4.0 Project Team & Roles

Project Leader	Priyanshu Mittal	❖ Manages the overall project and team. ❖ Liaises with users, IS Director, and technical units. ❖ Prepares and monitors project/phase plans, reporting progress and deviations. ❖ Manage resources, costs, and ensure work meets standards.
Technical Leader	Samarjeet Singh	❖ Acts as technical advisor to the Project Leader ❖ Prepares and oversees detailed technical plans. ❖ Ensures team members are trained in technical standards. ❖ Report on technical progress and evaluate modifications.
Quality Control Specialist	Aniket Thakur	❖ Tracks project effort, costs, and monitor activities. ❖ Ensures documentation and technical standards are followed. ❖ Coordinates project testing and evaluates user requirements. ❖ Supervises quality control team members.
Lead Programmers	Samarjeet Singh and Aniket Thakur	❖ Develops programming standards and review specifications. ❖ Build systems for analyst and designer specifications. ❖ Lead testing, walkthroughs and documentation. ❖ Supervises programming team members.
Leads System Analyst	Saif Alam	❖ Gathers and documents user requirements through interviews. ❖ Proposes solutions and conducts cost-benefit analysis. ❖ Designs data models, interfaces, testing/training plans. ❖ Prepares documentation.
Scrum Master	Priyanshu Mittal	❖ Coaches the team on Scrum principles and practice. ❖ Facilitates Scrum ceremonies. ❖ Protect the team from external distractions and scope creep. ❖ Encourages continuous improvement and collaboration.
Development Team	Saif Alam, Samarjeet and Aniket Thakur	❖ Cross-functional professionals who build the product. ❖ Self-organize to deliver increments each sprint. ❖ Estimate work, commit to sprint goals, and ensure quality. ❖ Share knowledge, stay cross-functional and own the results collectively.
Product Owner	Samarjeet Singh	❖ Represent client/stakeholder needs and define product vision. ❖ Owns and prioritizes Product Backlog. ❖ Translates needs to user stories and acceptance criteria. ❖ Accepts/rejects completed work to ensure value delivery.

5.0 Advisability Study

5.1 Evaluation Criteria

The selection of software, hardware, and tools for the RNAverse platform was guided by key factors to ensure efficiency, usability, scalability, and long-term sustainability.

- **Technical Integration Requirements:**
    1. Compatibility: All software components must work seamlessly with the pre-trained Python-based ML/DL models and integrate smoothly with the University of Winnipeg's existing ACS server infrastructure.
    2. Performance: The platform must handle batch processing of multiple RNA sequences efficiently, with prediction responses returning within reasonable timeframes even during concurrent user access.
    3. Maintainability: The architecture should support straightforward updates and modifications, allowing future research teams to add new methylation types or enhance existing models without major system overhauls.

- **User-Centric Requirements:**
    1. Usability: The interface must be immediately understandable to researchers with biological backgrounds but limited computational experience, requiring minimal training or technical support
    2. Accessibility: The system should be web-accessible through standard browsers without requiring special plugins or software installations on user machines.
    3. Learnability: The platform should allow users to quickly become proficient with minimal guidance, supported by clear instructions, examples, and help documentation.

- **Resource Management Requirements:**
    1. Cost-effectiveness: Priority given to open-source solutions and technologies available through university licensing to maintain zero budget requirements
    2. Sustainability**:** The selected tools and frameworks should have active community support, regular updates, and comprehensive documentation to ensure long-term maintenance and viability beyond the initial development phase.

5.2 Software Options and Comparative Analysis

The following software technologies were evaluated for developing the RNAverse platform based on their suitability for machine learning model integration, scalability, and usability.

5.2.1 Python + Django

- **Advantages:**
  Django is a full-featured Python web framework ideal for large-scale applications. It provides built-in support for data handling, routing, and security. Its native compatibility with Python's machine learning libraries (e.g., scikit-learn, TensorFlow, PyTorch) allows seamless integration of pre-trained ML/DL models into the backend.
- **Limitations:**
  Django's setup and architecture may be more complex for smaller or static projects.
- **Recommendation:**
  Recommended as the **primary backend framework** due to its scalability, modular design, and native support for Python-based ML models.

5.2.2 Node.js + Express

- **Advantages:**
  Provides high-performance handling for real-time data and has an extensive ecosystem of libraries.
- **Limitations:**
  Limited native support for Python-based ML/DL integration; requires additional layers or APIs to communicate with ML scripts.
- **Recommendation:**
  Suitable when a JavaScript-based backend is preferred or when real-time streaming is required, but **not ideal** for this ML-focused platform.

5.2.3 JavaScript (React / Angular)

- **Advantages:**
  React and Angular are powerful frontend libraries for creating dynamic, responsive, and user-friendly interfaces. They enable real-time updates and integrate efficiently with REST APIs for visualizations and result presentation.

- **Limitations:**
  These frameworks handle only the frontend logic and rely on backend APIs (e.g., Django) for computational tasks.
- **Recommendation:**
  React is recommended for the **frontend interface** due to its compatibility with Django REST APIs and its support for modular, interactive visualizations.

5.2.4 Database Options

- **Relational Database (SQL)**
  Examples**:** SQLite, PostgreSQL, MySQL
  Advantages:
  - Structured schema, easy to query
  - Supports relationships like **Sequence → Prediction → Model**
  - Simplifies CSV export for results
  - Good for future expansion or persistent storage

  Disadvantages:
  - Slightly more setup if using server-based DBs (PostgreSQL/MySQL).

- **NoSQL Database (Document-Based)**
  Examples**:** MongoDB
  Advantages:
  - Flexible schema, suitable for variable-length sequences
  - Fast read/write for temporary storage

  Disadvantages:
  - Querying complex relationships is harder
  - Overkill for structured prediction data

Recommendation:

- **SQLite** for Phase 1: Lightweight, no server setup, works seamlessly with Django/Flask
- **PostgreSQL** as anticipating multiple concurrent users or scaling in the future

Reasoning:

The relational structure fits the **Sequence → Prediction → Model** relationships, allows easy CSV export, and provides future scalability.

5.3 Tool Comparative Analysis

5.3.1 GitHub

- **Advantages:**
  Facilitates collaboration, version control, and code history tracking which is essential for maintaining reproducibility in research-based projects.

- **Justification:**
  Serves as the central repository for source code management, allowing all team members to track and merge contributions efficiently.

5.3.2 Jira

- **Advantages:**
  Supports task tracking, milestone management, and sprint planning for structured project progress monitoring.

- **Justification:**
  Enables hybrid Agile–Waterfall project management, aligning development tasks with academic project timelines and research goals.

5.3.3 Docker

- **Advantages:**
  Ensures consistent environments for development, testing, and deployment by packaging applications and dependencies into isolated containers.

- **Justification:**
  Docker simplifies the transition of the project between local machines and the University's ACS servers. It eliminates environment inconsistencies, making ML/DL integration and web deployment reproducible and stable.

5.4 Hardware and Hosting Considerations

The platform will initially be hosted on the **University of Winnipeg's ACS server**, which provides the necessary computational resources for web hosting and ML inference tasks.
For scalability, future deployment on cloud infrastructure (e.g., AWS or Google Cloud) may be considered if traffic or processing demand increases.

Hardware Justification:

- **CPU:** Multi-core processors ensure efficient model inference and simultaneous request handling.
- **RAM:** 16–32 GB provides adequate memory for pre-trained model execution and batch data handling.
- **Storage:** Needed for model files, logs, and intermediate results.
- **Network:** HTTPS and reverse proxy guarantee secure, stable communication.
- **Deployment:** Docker ensures reproducible environments and easy system maintenance.

5.5 Recommendations

- **Backend Framework:** *Python (Django)*
  Django is recommended due to its scalability, robust structure, and direct compatibility with Python-based ML/DL models. The Django REST Framework will be used to serve model predictions via APIs to the frontend.

- **Frontend Framework:** *React.js*
  React is chosen for building an interactive and responsive interface that allows sequence input, visualization of prediction results, and CSV download functionality

- **Database:** *SQLite / PostgreSQL*
  SQLite is recommended for Phase 1 because it is lightweight, requires no server setup, and works seamlessly with Django. PostgreSQL is suggested for future scalability if multiple concurrent users are expected.
  The relational structure supports the **Sequence → Prediction → Model** relationships and allows easy CSV export of results.

- **Development Tools:** *GitHub and Jira*
  GitHub ensures effective version control and collaboration. Jira supports structured task and milestone tracking aligned with project objectives.

- **Containerization and Deployment:** *Docker*
  Docker will be used to containerize both the backend and frontend applications. This ensures smooth deployment across local and ACS environments, minimizing configuration issues.

6.0 General Design

This part focuses on **system-level design** before Agile development begins.

6.1 Description:

The RNAverse platform is a modular, web-based system that allows researchers to submit RNA sequences and receive predictions about methylation status using pre-trained machine learning (ML) and deep learning (DL) models.

Main Components:

User Interface (UI): A web-based frontend that works with browsers and it offers an input form for RNA sequence submission and dashboards for viewing predictions, visual analytics of predictions and downloadable reports.

Backend Server: This will manage user requests, interact with ML/DL models, process uploaded data and will handle communication between the frontend and the database.

Pre-trained ML/DL Models: This includes Support Vector Machine (SVM), Random Forest (RF), and other algorithms that will be integrated into the backend for precise methylation prediction.

Database: This securely stores RNA inputs, prediction results and user information for future retrieval, analysis, and reporting.

Data Flow: When a user submits RNA sequences through the interface, the backend preprocesses the input by encoding features and validating the data. It then routes the information to the selected prediction models. After the models generate results, the predictions appear on the user's dashboard and are stored in the database for later access or download.

6.2 Context-Level Process Model:

External Entities:

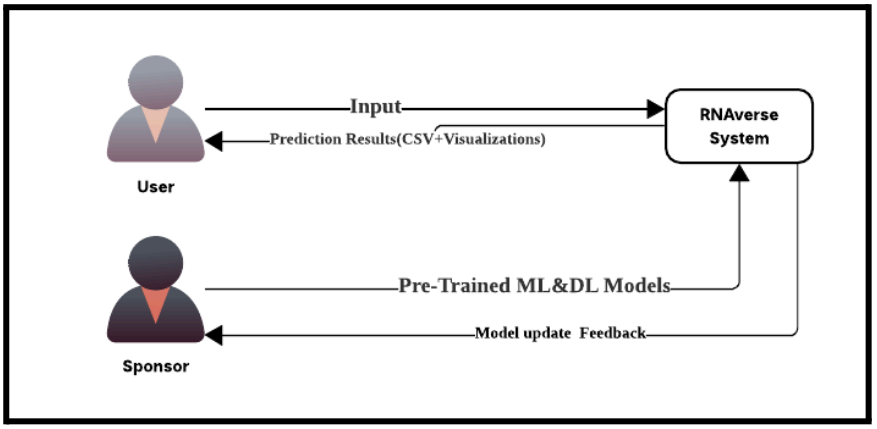
1. **Researcher / User**
   - Inputs RNA sequences into the system.
   - Receives prediction results (CSV downloads and visualizations).
2. **Sponsor**
   - Supplies pre-trained models (SVM, RF, CNN, etc.).
   - Provides updates to models if needed.

RNAverse System:

- Accepts RNA sequences from users.
- Validates input sequences.
- Processes sequences through pre-trained ML/DL models.
- Generates prediction results in both tabular and visual formats.
- Provides downloadable CSV output.
- Maintains logs and temporary storage for results.

Input / Output Flows:

- Input: RNA sequences (single or batch) from the user.
- Output: Predicted methylation results (table + chart) to user.



High Level Data Flow Diagram

6.3 Preliminary Data Model:

The preliminary data model for the proposed RNA methylation prediction system defines the key entities, their attributes, and relationships that support the flow of information from user input to prediction output.

The UserInput entity stores all RNA sequences submitted by the user. Each submission is assigned a unique *Sequence_ID* (PK)) , *RNA_Sequence* date and Sequence Length.

The Model entity maintains information about each pre-trained machine learning or deep learning model used for prediction. Each record is uniquely identified Model_ID, *Model_Name* and *Model_Type* attributes.

The Prediction_Results entity stores the output generated after the prediction process. It is uniquely identified by *Result_ID* (PK) and references both the *Sequence_ID* (Foreign Key→UserInput) and the *Model_ID* (Foreign Key→Model). This table includes attributes such as *PredictedMethylationType*, *Predicted_Status*), and *Probability_Score*.

The relationships between entities are as follows:

- User_Input→Prediction_Results: One-to-many relationship, as each RNA sequence can generate multiple prediction results using different models.
- Models→Prediction_Results: One-to-many relationship, since each model can create multiple prediction results for different sequences.
- User_Input ↔ Models (indirect relationship)**:** Together, they form a many-to-many relationship through the Prediction_Results entity, because each sequence can be evaluated by multiple models and each model can process multiple sequences.

Additionally, the system includes a comparison mechanism within the Prediction Result entity to evaluate outputs from all models. For each sequence, the system identifies the model that achieves the highest probability score and highlights that result in the final output and visualization. This ensures that users can easily determine which model provides the most confident prediction for each RNA sequence.

6.4 Description of Screens/Reports:

The interface for RNAverse is designed to provide researchers with an easy and simple user experience. This enables them to predict RNA methylation without requiring technical or programming skills. The web interface is transparent and intuitive; users can easily navigate data input, prediction computation, and presentation of results. Each page serves a particular purpose, but together they create an unbroken path from sequence submission to result interpretation.

- **Home Page:**
  The Home Page is the first screen users see when opening RNAverse. It briefly explains the platform's purpose of predicting RNA methylation sites using advanced machine learning and gives easy access to other sections. Users can see a short overview of RNAverse's features and find links to helpful resources and contact options. A clear "Predict Methylation" button takes users directly to the main input interface.

- **Prediction Page:**
  In this page, researchers can submit their RNA sequences to get predictions. They can either paste sequences directly into a text box or upload a FASTA file. Dropdown menus allow users to choose the type of methylation they want to analyze, such as m6A, m5C, or m7G. The system automatically checks for format errors and displays helpful messages if something is missing or incorrect. After the data is validated, users can click the "Predict Methylation" button to start the analysis.

- **Results Page with Download Option:**
  After submission, the system processes the data and directly displays the Results page. Here, users can view the predicted methylation types, associated models (e.g., CNN, XGBoost, Random Forest), and corresponding probability score with predicted sites. The Results Page includes a "Download CSV" button, allowing users to save their prediction outcomes and Interactive charts generated using Matplotlib help users easily interpret and explore the prediction outcomes.

- **Help and Documentation Page:**
  The Help and Documentation Page provides valuable information for new users. It includes a short instruction manual on formatting RNA sequences, descriptions of each model. It also offers some example sequences to guide users through the process. Additionally, there's a

FAQ and troubleshooting section that covers common input and interpretation issues. This resource makes RNAverse more accessible to general users.

- **Conclusion:**

  RNAverse's UI is designed to be simple, interactive, and scientifically clear. Through automatic data quality checks, continuous prediction updates, and organized reports downloadable with a click, researchers can easily turn raw data into valuable biological insights. The transparent display of results in tabular and visual forms makes predictions more comprehensible, reproducible, and credible for RNA methylation analysis.

6.5 Testing Plans (Overall):

Testing is a crucial part of the RNAverse's system development lifecycle which supports both the planned and documented approach. The aim of the testing phase is to verify correctness, usability and performance for all features provided by the RNAverse system.

Testing levels and strategies:

- **Unit testing:**
  1. Every core module such as input validation, file handling, data visualization will be tested independently.
  2. Python testing frameworks like pytest will be used to test script validation such as error handling, sequence format checks, output structure etc.
- **Integration testing:**

  Focus on validating the interaction between the modules such as:
  1. Data flow from the web frontend to the backend APIs.
  2. The backend system connects with pre-trained machine and deep learning models such as SVM, Random Forest, CNN etc to generate prediction results.
  3. Ensures consistent data storage and retrieval between backend components and the database.

- **System testing:**
    1. Comprehensive end to end testing will simulate real user interaction, project uploads, model selection, predicting generation, visualization and data export to CSV.
    2. The Objective is to confirm that RNAverse functions smoothly and aligns with the requirements in the project proposal.
- **Performance testing:**
  This testing will allow the platform's responsiveness, stability and scalability under workloads, specifically when handling sequence submissions.
- **User Acceptance Testing (UAT):**
  Conducted by team members and test users from the University of Winnipeg to confirm usability and output correctness.

6.6 Conversion Plans:

As RNAverse is a new system, no data migration or legacy system conversion is required. However, the following steps will ensure smooth deployment:

- Deploy Docker containers on the ACS server to replicate the tested local environment.
- Transfer pre-trained ML/DL model files and configurations to the model repository directory on the server.
- Conduct post-deployment verification testing to confirm all endpoints and visualizations function correctly.

6.7 Training Plans:

Training will be simple and user-focused, providing step-by-step guidance on submitting RNA sequences, selecting models, and interpreting results. Clear examples will help researchers with limited programming experience use the platform confidently. So, the training will focus on being simple to follow, step by step.

- User Documentation: A complete User Guide will explain all the main features such as how to upload RNA sequences, understand the prediction results and download them.
- Interactive Help page: The Help page in RNAverse will give tips about correct sequence formats, common mistakes. It will also include an FAQ and troubleshooting section to solve common problems.

7.0 Agile Development Stage

The project follows a Hybrid Development Model that combines the structured planning strength of the Waterfall approach with the flexibility of Agile. Early phases such as requirement gathering, system analysis, and high level design follow a waterfall model. Actual development, integration, and testing cycles are carried out in Agile sprints. This provides strong documentation while still enabling iterative development.

7.1 Timeline and Sprints

Sprints	Timelines	Deliverables
Sprint 1	*November 10 to November 23, 2025*	Finalized functional and non functional requirements Architecture confirmation Repository setup Initial Backlog items
Sprint 2	*November 24 to December 7, 2025*	RNA input module with multi sequence support Validation and error handling Initial preprocessing pipeline Unit testing for input components
Sprint 3	*December 8 to December 21, 2025*	Completed formatting and preprocessing pipeline. Integration of SVM, RF, LR, and CNN models. Basic prediction workflow.
Sprint 4	*January 6 to January 23, 2026*	Integration of CNN and XGBoost models Visualization module with charts and plots Updated UI to display prediction outputs
Sprint 5	*January 23 to February 2, 2026*	CSV export functionality Optimization of preprocessing and inference flow. User testing feedback

Sprint 6	*February 3 to February 27, 2026*	Full integration testing. Error resolution and performance tuning. Documentation update. Sponsor review and feedback.
Sprint 7	*February 27 to March 16, 2026*	Pre deployment validation using sample datasets. User manual and help guide Deployment on ACS server Final Demonstration

7.2 Agile (Scrum) Implementation

- **Sprint Planning:** Before each sprint starts, the team reviews the product backlog and selects priority tasks. Tasks are estimated, assigned, and organised into technical subtasks. Sprint goals are finalized based on available time and model readiness.

- **Daily Stand-ups:**The team conducts a short stand up meeting every two days. Each member reports:
    - Completed work
    - Any Backlog
    - Work planned for the day
    - Any blockers or issues

  Daily standups ensure clear communication, fast problem solving and steady progress.

- **Sprint Review:** At the end of each sprint, the team presents the working increment to sponsors Dr. Liu and Dr. Ramanna. Demonstrations may include input forms, prediction results, visual outputs, or integration of new trained models.

- **Sprint Retrospective:** The team meets internally to evaluate:
    - What went well.
    - What problems occured
    - What improvements will be applied in the next sprint

  This strengthens team workflow and overall project efficiency.

7.3 Agile (Scrum) Artifacts

- **Product Backlog:**

  Required System Features:

  - *Sequence Input Module*
  - *Data Preprocessing*
  - *Model Integration for each methylation type*
  - *Visualization of prediction results*
  - *CSV Export*
  - *Admin Features*
  - *Future Model Additions*

- **Sprint Backlog:**

  A selected subset of the product backlog dedicated to the current sprint. It contains tasks that the team commits to completing within that sprint.

- **Increment:**

  The increment is the working version of the system at the end of each sprint. Each increment includes new features and improvements while keeping previously built features stable and functional.

7.4 Supporting Practices

- Task Tracking: The team will use **JIRA** for task assignment, Sprint boards, backlog management, and progress monitoring.
- Version Control: All work is maintained on **GitHub.** Branching, pull requests and code reviews ensure clean integration.
- Charts for Progress:

  Velocity charts are Burndown charts will be used to measure team workload progress and Sprint completion rates.

7.5 Continuous Integration and Testing

Continuous integration ensures new code is combined with the main system without breaking existing features.

The team will use

- GitHub branches for new features.
- Pull requests with code review.
- Testing after each merge.

Testing includes

- Unit testing for modules such as pre processing
- Integration testing.Our prediction workflow
- User interface testing,
- Model output validation with sample sequences.

GitHub Actions, or manual integration tests will be used to ensure quality at every stage.

8.0 Project Management

8.1 Assumptions

Technical Assumptions

- The pre-trained ML/DL models provided by the sponsor will execute successfully in the target environment without major reconfiguration.
- All required Python libraries are compatible with the server and version requirements.
- The web framework can be deployed on the University server.
- The project's database will be accessible throughout development.
- Dataset files are pre-cleaned, correctly formatted, and available for immediate use.
- All users will access the system through modern browsers with JavaScript enabled.
- Internet connectivity will remain stable for pulling dependencies and datasets from repositories.

Data & Model Assumptions

- The RNA methylation dataset used for predictions is provided by the sponsor and approved for academic use.
- The pre-trained models provided by the sponsor) achieve sufficient accuracy for demonstration purposes, no re-training is required during the PPR phase.
- Future versions may include retraining or model updates with new RNA data.
- Model prediction times will remain within an acceptable range.
- Input RNA sequences are assumed to be valid and properly formatted by the user.

Operational / Infrastructure Assumptions

- All team members will have access to the same development environment and shared Git repository.
- Sufficient compute resources are available for running ML predictions.
- The system will primarily run on desktops/laptops, no mobile optimization is expected.
- Backups and version control are handled automatically through GitHub commits.
- The deployment environment will remain accessible until project evaluation and demo**.**

Management / Collaboration Assumptions

- The team will follow the hybrid (Agile + Waterfall) structure initial design locked early, later iterations handled through Agile sprints.
- All sprint meetings, demos, and reviews will happen on schedule.
- The sponsor will provide clear, timely feedback during milestone reviews.
- The IS Director will remain available for consultations and approvals.
- Meetings with sponsors or instructors will happen as per agreed schedules.
- Team communication tools will remain accessible for coordination.
- No major personnel changes or absences will affect delivery timelines.
- Each team member will complete their assigned sprint tasks on time.
- All team decisions will be recorded during sprint retrospectives
- The project timeline will remain aligned with the University's term schedule without unexpected breaks.

8.2 Change Control Management

All change requests, whether from team members or stakeholders, will be formally documented and submitted through a shared document or GitHub. Each request will include the reason, impact, and proposed solution. The project team will assess the effect on scope, timeline, resources, and quality before proceeding. Minor changes, such as UI updates, may be approved by the Team Lead. While major changes, including architecture or model updates, require approval from the Project Sponsor and IS Director. Once approved, changes will be recorded with full details, assigned to the appropriate sprint or milestone, and communicated to all team members during meetings to ensure transparency, coordination, and proper version control.

8.3 Maintenance Issues

After deployment, the system will be maintained through regular bug fixes, user feedback monitoring, and periodic updates of ML/DL models with new RNA methylation data. Future improvements may include adding new methylation types, enhancing visualizations, and refining the user interface. Version control will continue through GitHub to track all updates and ensure smooth collaboration. Regular compatibility checks will be performed to keep the web application aligned with university servers and updated software libraries. The system is designed for future teams or research assistants to extend it by adding datasets, improving model accuracy, or integrating additional bioinformatics tools.

8.4 Project Plan

Project Milestone Summary

2025-2026

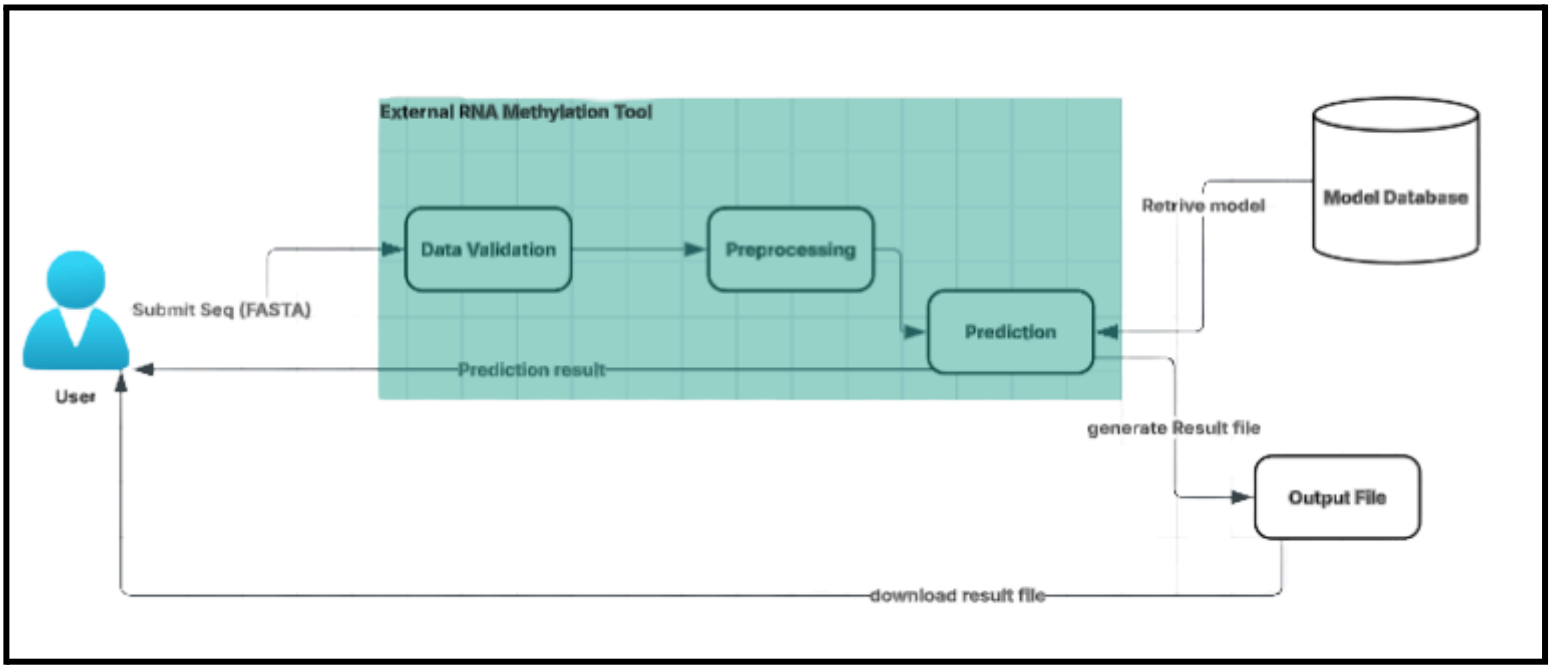
Phase / Task	Milestone	Target Deadline	Deliverables
Team Formation & Role Assignment	Project Initiation	Week of Sept 2, 2025	Team role list, collaboration setup
Initial Sponsor Meeting	Requirement Discussion	Week of Sept 8, 2025	Finalized requirements, dataset access notes
Project Proposal Submission	Proposal Approval	Week of Sept 26, 2025	Approved project proposal document
PPR Submission & Plan Review	Design Phase	Week of Nov 10, 2025	Project Planning Report, system design diagrams.
Sprint 1–2: Input & Preprocessing Module	Development Phase	Nov 10 – Dec 2025	Input validation and preprocessing pipeline
Mid-Project Review	Progress Evaluation	Week of Jan 12, 2026	Prototype demo with sponsor feedback
Sprint 3–4: Model Integration	Development Phase	Jan 6 – Jan 23, 2026	Integrated ML models (SVM, RF, LR, CNN) connected to frontend
Sprint 5–6: Visualization & Output Module	Feature Completion	Jan 23 – Feb 27, 2026	Visualization dashboard, CSV export, performance tuning
Development Review	Testing Preparation	Week of Feb 27, 2026	Test plan, integration report
Sprint 7: Integration & Testing	Final Integration	Feb 27 – Mar 16, 2026	Perform full system integration, final testing, and documentation updates
Final Turnover & Sign-Off	Deployment	Week of Mar 16, 2026	Deployed system on ACS server, signed approval
Seminar & Final Demonstration	Project Presentation	March 27, 2026	Conduct final live demo and seminar for faculty and sponsors.
Completion Checklist	Project Closure	Week of April 06, 2026	Submit final deliverables, report, and repository archive.

9.0 Budget and Resources

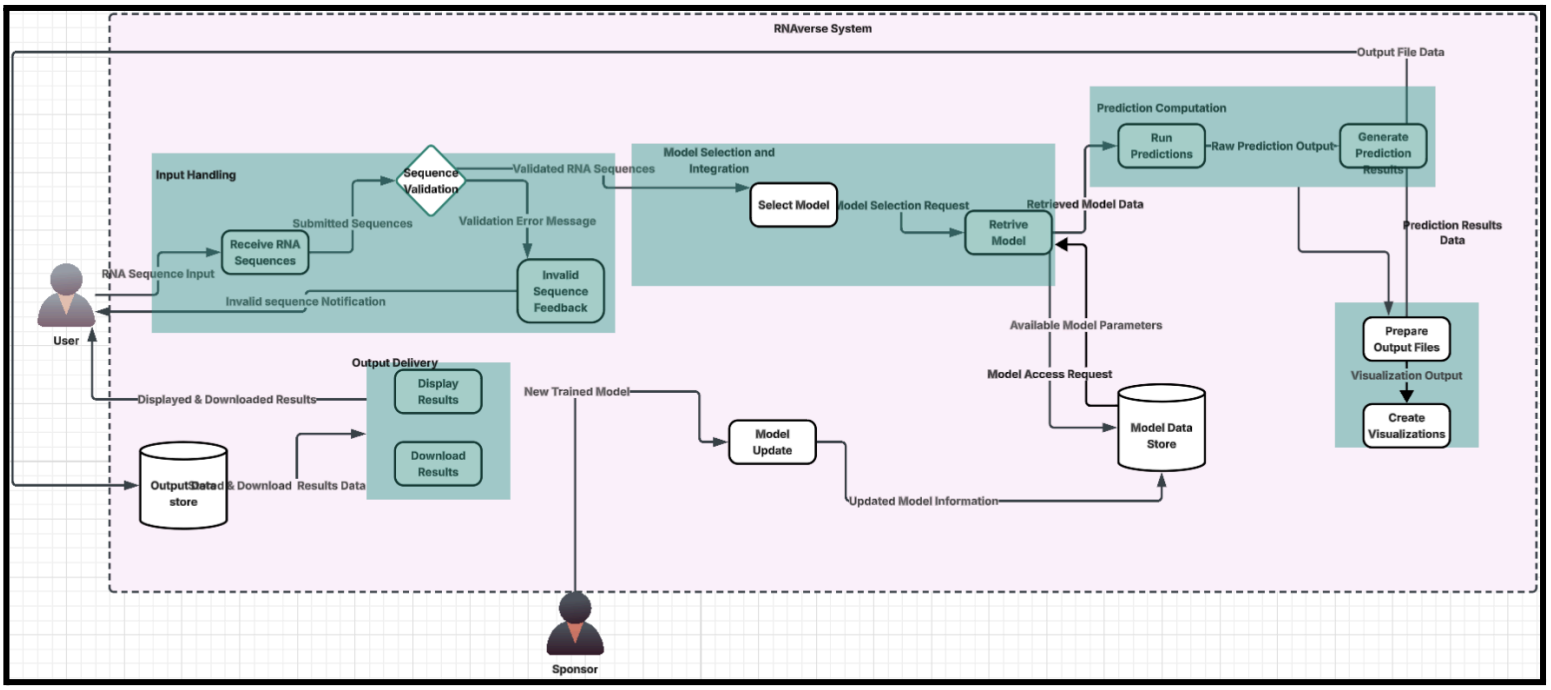
The project uses university servers and open-source tools, so there are no major costs. Team members contribute their time and effort for design, development, testing, and deployment using free resources.

10.0 Appendix

A. DFD for External System:

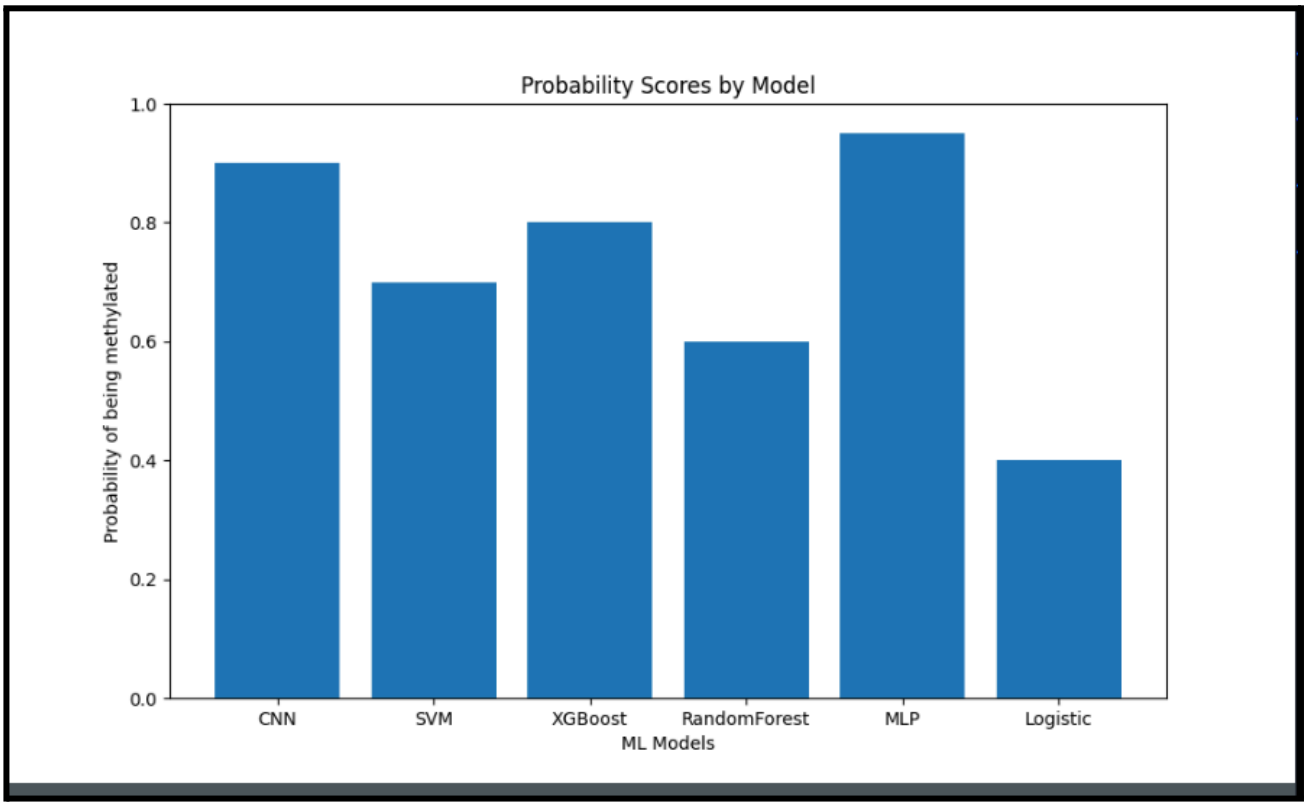


B. DFD for Proposed System:



C. Report designs

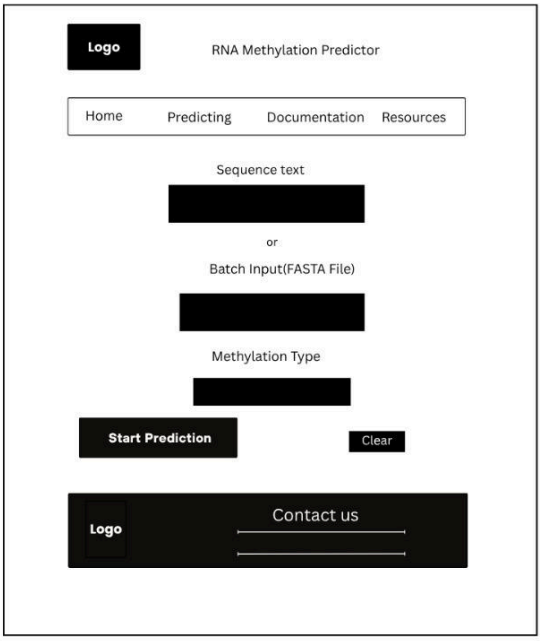
SEQ ID	RNA SEQUENCE	PREDICTED METHYLATION TYPE	MODEL USED (HIGHEST PROBABILITY)	PROBABILITY SCORE	STATUS
Seq1	AGCUAGCU	m6A	CNN	0.94	methylated
Seq2	UAGGCUAC	m7G	XGBoost	0.91	methylated
Seq3	CCGUAUAG	m5C	Random Forest	0.89	methylated



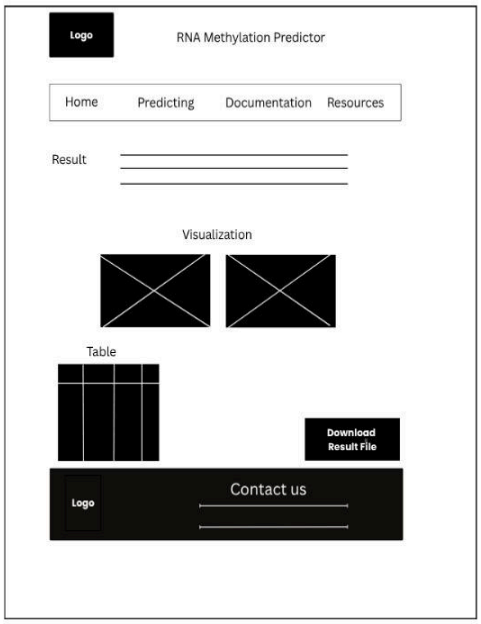
D. Screen Design Wireframes

Home Page

Predicting Page



Result Page



Help & Documentation Page

E. Work request form

Work Request Form

Please fill out the details to request and approve changes to the project.

Begin →

Requested By *

Name

Date *

Description of Work *

Priority *

A Low

B Medium

C High

Expected Completion Date *

Specify sprint or date

Assigned To

Team member name

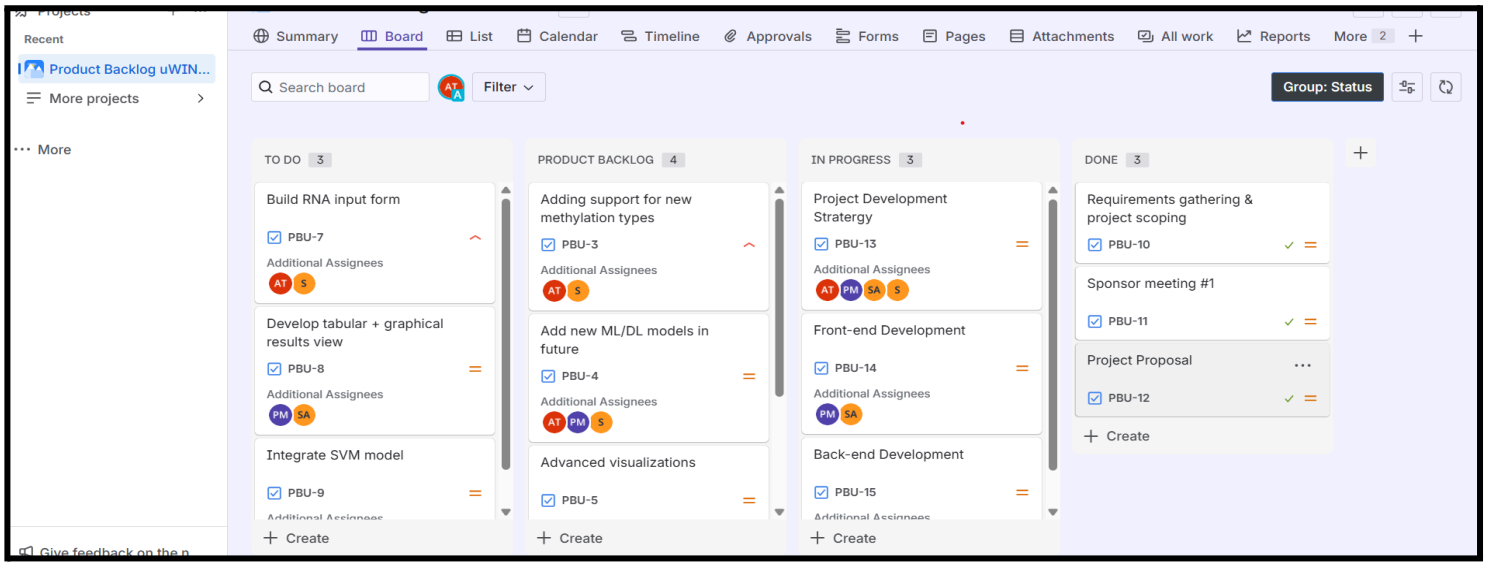
Remarks / Notes

Submit Work Order

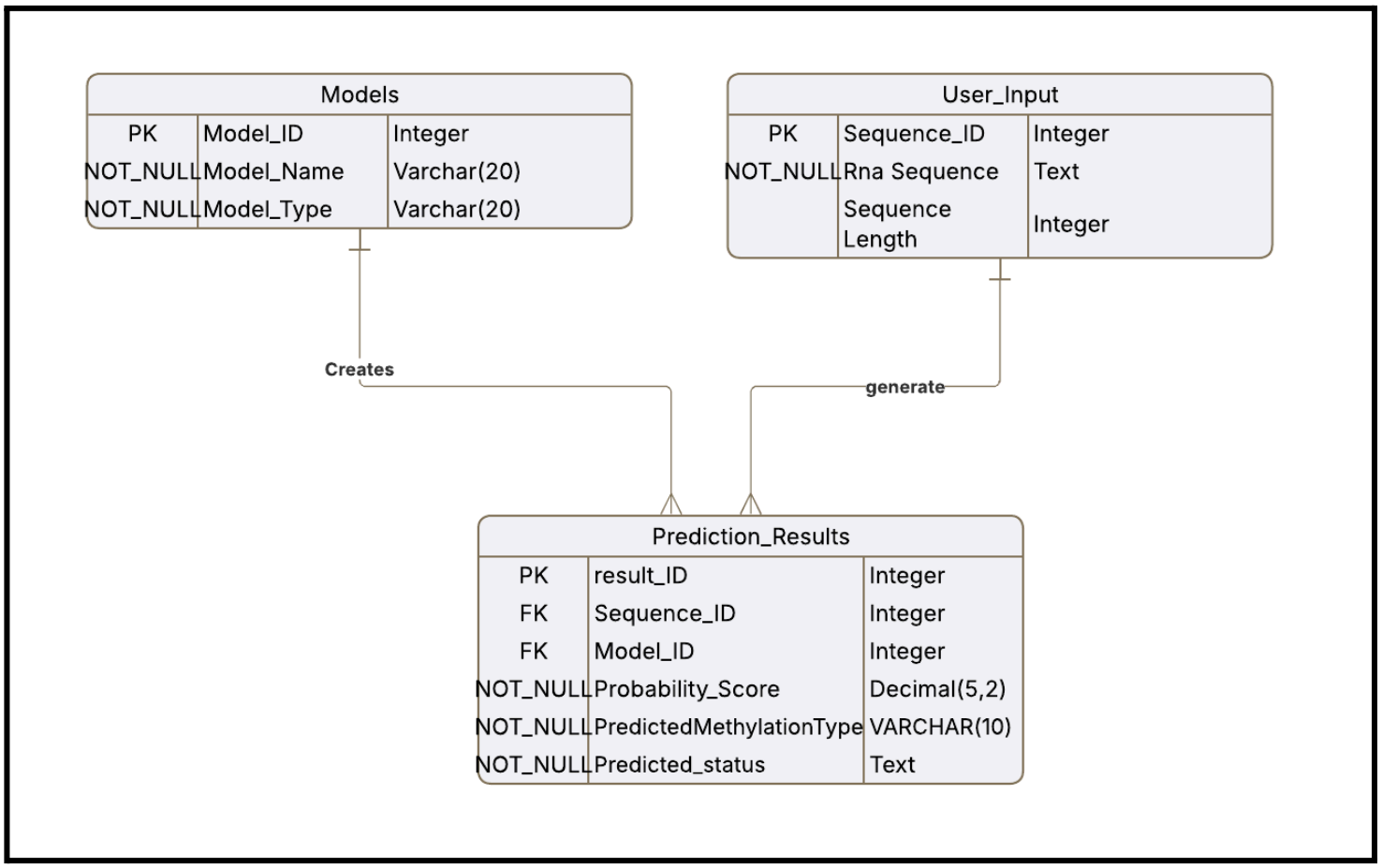
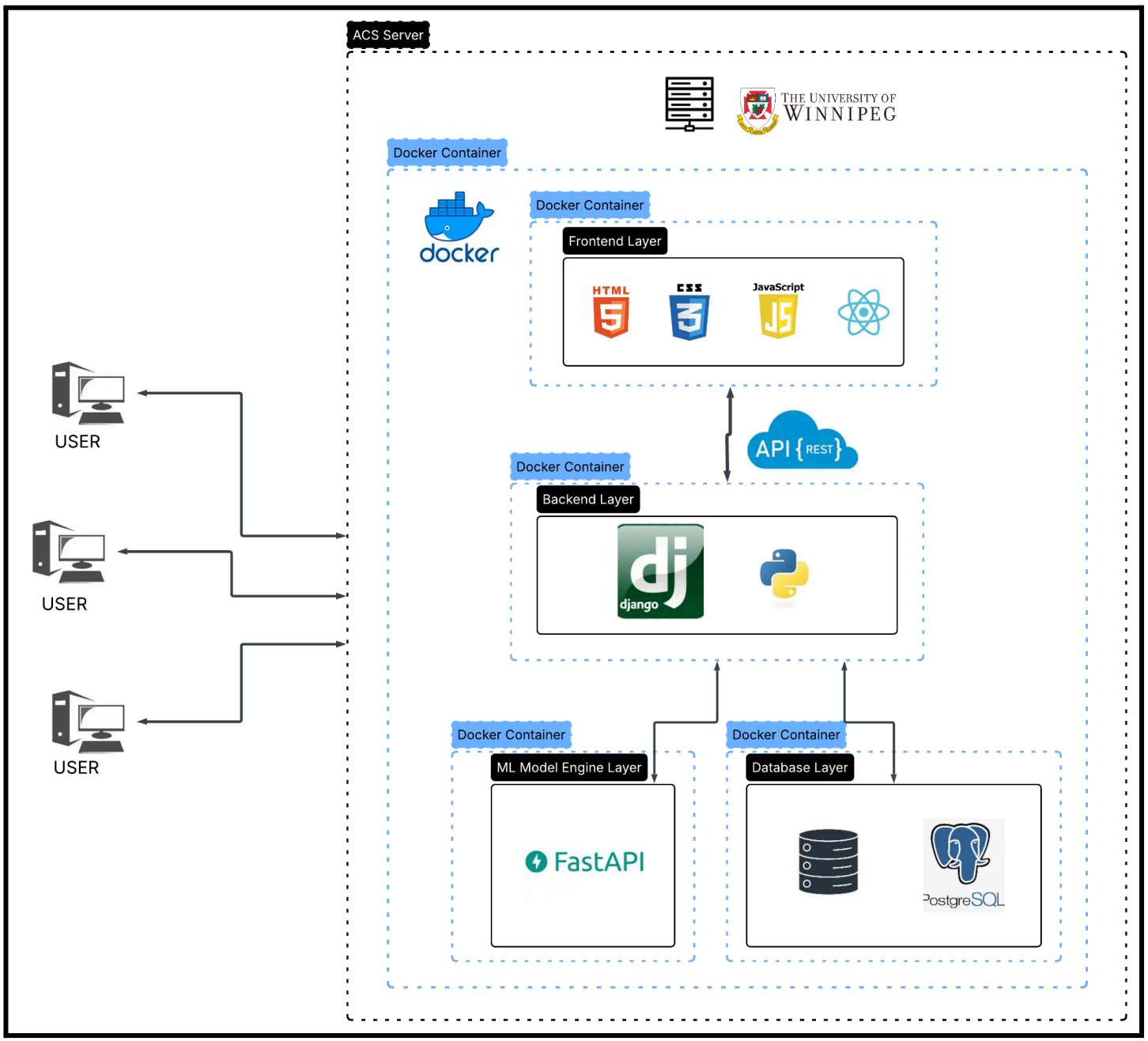
Work Request Submitted

Thank you for submitting the change request. The project team will review and follow up accordingly.

F. JIRA snapshot:



G. ERD


Models
PK Model_ID Integer
NOT_NULL Model_Name Varchar(20)
NOT_NULL Model_Type Varchar(20)
User_Input
PK Sequence_ID Integer
NOT_NULL Rna Sequence Text
Sequence Length Integer
Creates
generate
Prediction_Results
PK result_ID Integer
FK Sequence_ID Integer
FK Model_ID Integer
NOT_NULL Probability_Score Decimal(5,2)
NOT_NULL PredictedMethylationType VARCHAR(10)
NOT_NULL Predicted_status Text

H. Architecture Design Blueprint